Weekly Student Progress Performa

Team Id	CS26084	**Week no / Date**	Week 5 17 August 2026
Project Title	SentinelCore: A Network Detection and Incident Response Platform		
Members Present	Om Rashiya (24cs084), Yash Patel (24cs082)		
Mentor	Akshita Kadam		

WEEK 5 • Authentication, RBAC, and Asset Workspace

Section	Details to be filled by the team
Planned Tasks (from last week)	Stabilize the Docker stack and implement secure authentication, role-based access, session refresh, and audit logging.
Tasks Completed This Week	Completed the container stack health flow, added Argon2 password hashing, JWT access tokens with HttpOnly refresh cookies, login throttling, seeded admin support, protected React routes, role-aware navigation, and audit-log services.
Commits Made	Total: 1 milestone commit. 9fc44f2 — “M0: Docker stack complete, health endpoint working”. Authentication work continued in the later consolidated M1–M5 commit.
Issues Created / Closed	Created: Access tokens must not persist in localStorage; repeated login attempts require throttling. Closed: Kept access tokens in memory, used rotating refresh cookies, and added server-side throttling and role checks.
Milestone Status	~36% complete — on track. Secure access control and the deployment foundation were functional.
Blockers / Challenges Faced	Maintaining a session without exposing durable tokens to JavaScript storage. The refresh endpoint and in-memory access-token model resolved this.
Plan for Next Week	Implement asset discovery, scan parsing, device inventory, and the privileged helper protocol.

Research and Technical Study

Research objective. Stabilize the Docker stack and implement secure authentication, role-based access, session refresh, and audit logging.

Study completed. Studied password hashing, short-lived bearer tokens, refresh-token cookies, RBAC enforcement, and tamper-evident audit design. Authorization is checked again in the backend even when navigation is hidden in the UI.

Authentication and authorization controls

Control area	Implemented choice	Website effect
Password storage	Argon2 hashing	Credentials are verified server-side
Access session	Short-lived JWT in memory	No durable access token in localStorage
Refresh session	HttpOnly refresh cookie	Session renewal is isolated from page scripts
Authorization	Viewer / analyst / admin RBAC	Protected routes and role-aware actions
Abuse handling	Login throttling and audit	Repeated attempts are controlled and recorded

Decision carried into the project

Completed the container stack health flow, added Argon2 password hashing, JWT access tokens with HttpOnly refresh cookies, login throttling, seeded admin support, protected React routes, role-aware navigation, and audit-log services.

Implementation Record and Website Evidence

Authentication flow

Implemented password verification, short-lived access tokens held in memory, refresh handling through an HttpOnly cookie, logout, and bootstrap administrator support.

Role enforcement

Protected React routes and adjusted navigation for viewer, analyst, and admin roles, while retaining backend authorization as the controlling check for every protected action.

Abuse controls and audit

Added login throttling and audit services so authentication attempts and state-changing actions can be recorded without relying on the browser.

Asset workspace preparation

Built the role-protected inventory surface with search, active-only and open-port filters, refresh, scan entry point, and the empty-state instruction shown in the evidence.

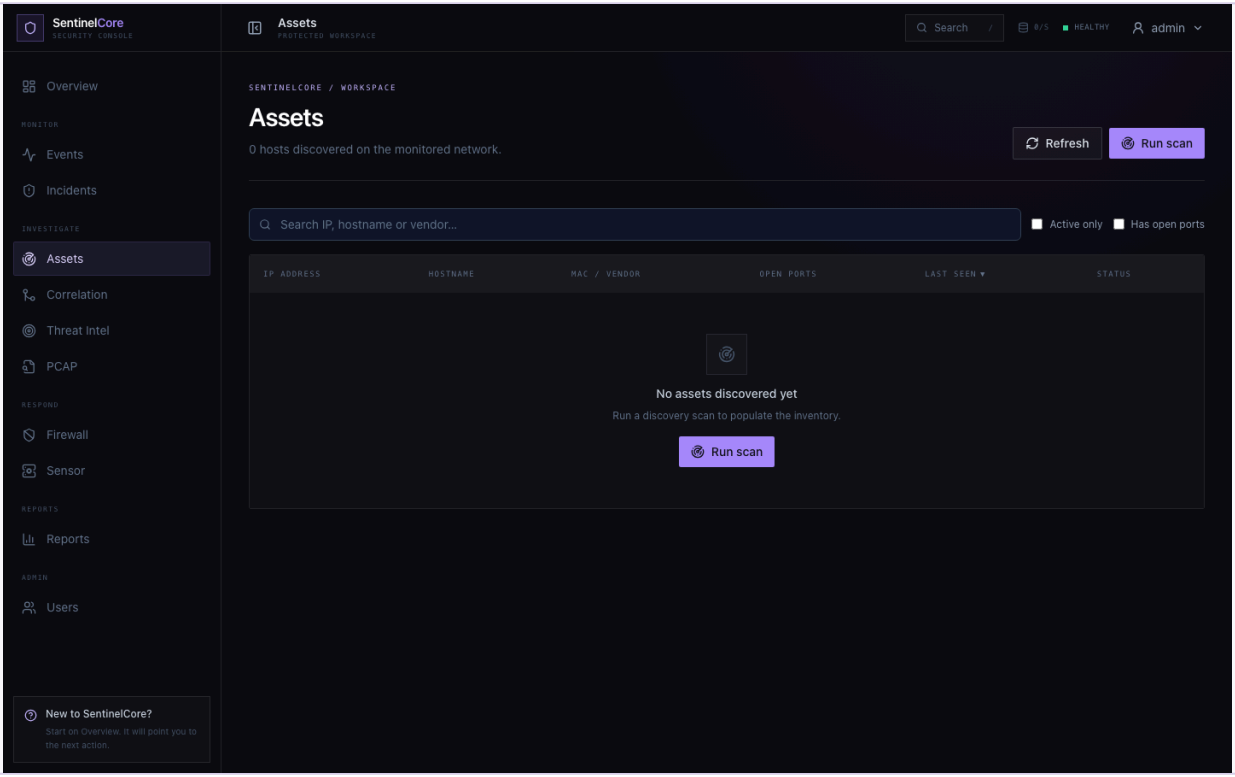


Figure W5.1 — Project-owned SentinelCore interface mapped specifically to Week 5.

Evidence interpretation. The Assets workspace shows inventory controls, scan access, active/open-port filters, and a clear empty state directing the operator to run discovery. Its visible controls, labels, status fields, and navigation provide direct evidence for the work described on this page.

Repository and Architecture Evidence

Authentication context implementation

frontend/src/auth/AuthContext.jsx

```
/** Refresh this many ms before the access token actually expires. */
const REFRESH_LEAD_MS = 60_000;

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  // The access token lives in a ref, not state: the API client reads it
  // synchronously on every request, and re-rendering the whole tree each time
  // it rotates would be pointless churn. It is never persisted anywhere.
  const tokenRef = useRef(null);
  const refreshTimer = useRef(null);

  const clearRefreshTimer = useCallback(() => {
    if (refreshTimer.current) {
      clearTimeout(refreshTimer.current);
      refreshTimer.current = null;
    }
  }, []);

  const clearSession = useCallback(() => {
    tokenRef.current = null;
    setUser(null);
    clearRefreshTimer();
```

Repository evidence captured directly from the SentinelCore workspace

Figure W5.2 — Authentication context implementation. This is a separate internal artifact, not a copy of the website image.

Repository files used for traceability

- frontend/src/auth/AuthContext.jsx
- frontend/src/routes/ProtectedRoute.jsx
- frontend/src/pages/assets/AssetsPage.jsx
- backend/app/api/routes/assets.py

Verification

The access model and asset controls were checked against the actual routes, frontend labels, authentication context, and the repository test coverage.

Traceability conclusion. Authentication context implementation directly supports the Authentication, RBAC, and Asset Workspace milestone. The access model and asset controls were checked against the actual routes, frontend labels, authentication context, and the repository test coverage. The artifact, source paths, implementation record, and website evidence therefore describe the same weekly scope.

Weekly Closure and Faculty Assessment

Completed outcome

Completed the container stack health flow, added Argon2 password hashing, JWT access tokens with HttpOnly refresh cookies, login throttling, seeded admin support, protected React routes, role-aware navigation, and audit-log services.

Commit record

Total: 1 milestone commit. 9fc44f2 — “M0: Docker stack complete, health endpoint working”. Authentication work continued in the later consolidated M1–M5 commit.

Issue and resolution

Created: Access tokens must not persist in localStorage; repeated login attempts require throttling. Closed: Kept access tokens in memory, used rotating refresh cookies, and added server-side throttling and role checks.

Milestone position

~36% complete — on track. Secure access control and the deployment foundation were functional.

Challenge handled

Maintaining a session without exposing durable tokens to JavaScript storage. The refresh endpoint and in-memory access-token model resolved this.

Next-week hand-off

Implement asset discovery, scan parsing, device inventory, and the privileged helper protocol.

Repository reference

https://github.com/rashiyaom/SentinelCore

Assessed By (Faculty)	Marks Awarded (5 marks)	Faculty Remarks